

Domain-Partition Algorithm for Turbomachinery Blades

A Block-Structured Meshing Pipeline

Domain-Partition 3D Project

July 9, 2026

Domain-Partition Algorithm for Turbomachinery Blades

A Block-Structured Meshing Pipeline

Domain-Partition 3D Project

July 9, 2026

T1_9 Turbine Hub — Overview

- T1_9 turbine hub: cylindrical surface at constant radius
- Input: T1_9_hub_raw.stl (raw triangulated surface)
- Goal: extract quadrilateral block structure for CFD meshing

T1_9_hub.png

Placeholder: 3D hub visualization

Pipeline: STL → unwrap → cross-field → singularities → streamlines → blocks

Unwrapped Hub Surface — Cylindrical Mapping

$$\theta = \arctan\left(\frac{y}{x}\right), \quad s = r \cdot \theta, \quad t = z$$

- **Isometric mapping:** distortion-free, preserves triangulation
- **Domain becomes rectangular** with a blade-shaped hole
- **Periodic θ -boundaries** (pitch = $\pi/4$ for 8 blades)

Code: `scripts/stage1/unwrap_surface.py:8-10`

$$c(x) = (\cos 4\theta(x), \sin 4\theta(x))$$

- **4-RoSy field:** invariant under 90° rotation
- **Representative vector** encodes direction modulo $\pi/2$
- **Boundary-aligned** via Dirichlet conditions

Literature: Knöppel et al., “Globally optimal direction fields,” SIGGRAPH 2013

Code: `scripts/stage1/partition_surface.py` — cross-field solver pipeline

Domain-Partition Algorithm for Turbomachinery Blades

A Block-Structured Meshing Pipeline

Domain-Partition 3D Project

July 9, 2026

Cross-Field $\rightarrow$ Representative Vector

- Each cross has 4 symmetric representatives (rotated by $\pi/2$)
- Pick the one closest to the desired direction
- Enables streamline integration along smooth field lines

Vector mapping:

$$v(x) = \arg \max_{c \in \text{cross}(x)} \langle c, d(x) \rangle$$

where $d(x)$ is the desired direction.

Citation: Knöppel et al., "Globally optimal direction fields," SIGGRAPH 2013.

$$\Delta\theta = 0 \quad \text{s.t.} \quad \theta|_{\partial\Omega} = \theta_{\text{wall}}$$

- **Solve harmonic Laplace** for a smooth angle field inside the domain
- **Dirichlet BC**: boundary values align with wall cross-vectors (4-RoSy)
- **Normalize** to extract the 4-RoSy representative ($\bmod \pi/2$)

Solver:

`tools/framefield.py`

Ref: Kneppe et al., "Globally optimal direction fields", SIGGRAPH2013

$$\text{index}(p) = \frac{1}{2\pi} \oint_{\partial B(p)} d\theta$$

- **4 tip singularities**, each with index $-\frac{1}{4}$
- **Poincaré-Hopf**: sum of indices equals Euler characteristic χ
- **Periodic cylinder**: $\chi = 0$, so indices must balance

$$\sum_i \text{index}(p_i) = \chi(\Omega)$$

Detection: `tools/singularity_detector.py` *Lit*: Kowalski 2015, “Blocking piecewise-linear domains”

$$p_{n+1} = p_n + \frac{1}{2}(k_1 + k_2)$$

$$k_1 = h v(p_n), \quad k_2 = h v(p_n + k_1)$$

- **Integrate** along representative vectors emanating from singularities
- **RK-Heun**: 2nd-order accurate, stable near singularities
- **Terminate** on boundary, singularity, or c_0 corner

Code: `StreamlineGenerator_v2.runge_kutta_heun_integrate_streamline` *Lit:*
Kowalski 2015, “Blocking piecewise-linear domains” (Prop. 9)

Streamline Simplification — Merging & Splitting

Angle criterion (Xiao Eq. 9)

$$120^\circ \leq \alpha \leq 240^\circ \text{ (3-valent)}, \quad 144^\circ \leq \alpha \leq 216^\circ \text{ (5-valent)}$$

Blend formula (Xiao Eq. 10 / Kowalski Eq. 28)

$$\gamma_{12}(s) = (1 - s) \gamma_1(s) + s \gamma_2(s)$$

- Merge nearby streamlines at singularities (angle criterion)
- Split at transversal crossings (T-junctions)
- Result: clean quad-only block boundaries

Code: `StreamlineMerging`, `StreamlineIntersectionSplitter` *Lit:* Xiao et al. 2020,
“Robust topology decomposition of 2D domains” (Alg. 2)

- **Collapse 3-sided seam wedges** (Xiao-style simplification)
- **Master-slave seam conformity** (DLR Sauer/Morsbach 2023 sec 2.7)
- **Result:** periodic block structure with conforming seam

Code: collapse_seam_wedges, symmetrize_seam_junctions in
tmesh_partition.py:332-488 *Lit:* Sauer/Morsbach 2023, “Automated blocking for
turbomachinery”, sec 2.7

$$\begin{aligned} X(u, v) = & (1 - v) \cdot S(u) + v \cdot N(u) \\ & + (1 - u) \cdot W(v) + u \cdot E(v) \\ & - \left[(1 - u)(1 - v) \cdot c_{00} + u(1 - v) \cdot c_{10} \right. \\ & \left. + (1 - u)v \cdot c_{01} + uv \cdot c_{11} \right] \end{aligned}$$

- **Bilinear interpolation** from 4 boundary curves
- **Guarantees C^0 continuity** at block edges
- **Cell counts solved conforming:** opposite sides match

Code: `tmesh_partition.py:_coons` (lines 881-892) *Lit:* Coons 1967, “Surfaces for Computer-Aided Design”

- **3 passages tiled:** shifts $-\text{pitch}$, 0 , $+\text{pitch}$
- **Interior seams:** grid points of neighboring copies coincide (CFD-conforming)
- **Visual check:** master-slave conformity confirmed by overlay

Code: `tmesh_partition.py:1271-1312 (plot_tiled)` *Lit:* DLR Sauer/Morsbach 2023, sec 2.7 master-slave